Payment-State-Aware Root Cause Analysis for Cascading Failures in Event-Driven Payment Processing Pipelines

Sarvadnya **[TODO: full author list, affiliations]**
[TODO: institution]

***Abstract*—We present ClearFlow, a live, running 8-service event-driven ISO 20022 payment pipeline instrumented so that transactional state – liquidity reservations, AML holds, idempotency-key collisions, and settlement finality – is queryable per payment, and a fault-injection harness that triggers *real* faults against this running system (process crashes, real AML holds, real idempotency collisions) rather than synthesizing labeled ground truth. Existing microservice root-cause-analysis (RCA) methods reason over generic telemetry and topology and do not model this payment-specific state as a causal factor. We evaluate a payment-state-aware RCA method against a telemetry/topology baseline, a naive telemetry-only baseline, and a general-purpose LLM given identical topology/telemetry evidence, using AC@k with Wilson confidence intervals on a held-out set of 25 live incidents disjoint from the incidents used to develop the method. The payment-state-aware method reaches AC@1 0.60 (95% CI 0.41–0.77) versus 0.48 and 0.44 for the topology and telemetry baselines respectively, and is statistically significant against a random-guessing floor ($p = 0.0034$); the LLM baseline, given equivalent evidence, scores 0.28 – below both non-LLM baselines. An ablation isolates which mechanism drives this result rather than reporting it as a black box: the six payment-domain-specific state fractions (AML hold, liquidity dwell, idempotency, settlement failure, validation stall) alone match the full method exactly (0.60), while a generalized process-crash completion-stall signal added to extend coverage to non-payment-domain faults contributes nothing extra and, used alone, underperforms even the topology baseline (0.40). The validated claim is accordingly narrower than the headline number: payment-domain transactional state, not process-crash-derived evidence, is what this evaluation supports as a causal RCA signal, and this is also why the method's dev-set improvement on confounded incidents does not replicate in the held-out set ($n = 3$) – a limitation reported directly rather than omitted. Two implementation-level findings generalize beyond this system regardless: a single-snapshot state flag is not specific evidence of a stuck fault without a dwell-time gate, and a process-crash fault produces no domain-state anomaly at all, requiring evidence framed as "did this payment reach the next stage's completion event," not as a state value – even though, on this evaluation's own evidence, that reframing alone is not sufficient to recover accuracy on crash-based faults. A direct evidence-vs-reasoning experiment gives a clean answer to why the LLM-based methods trail the formula: a static LLM given the identical G0–G3 payment-state evidence the formula uses scores exactly the same as one given only G0–G2 evidence (0.28 vs. 0.28, zero net change) – richer evidence alone does not help a static LLM. An agentic variant given real tool access to the live system instead of a static summary reaches 0.44–0.52, more than double the static baseline, from the same underlying information but a reasoning mode able to actively decide what to inspect. Separately, an interventional experiment no passive-dataset methodology in this literature can run – triggering the identical fault under two different real, controlled payment-state backgrounds – finds no detectable difference in cascade propagation at the sample size tested ($n = 4$–5), reported as an honest null result with its own limitations stated rather than omitted or oversold.**

***Index Terms*—root cause analysis, cascading failures, event-driven architecture, payment systems, microservices, observability**

I. Introduction

Payment processing pipelines are distributed systems with a property most microservice architectures do not have: every request carries *transactional state* that persists and constrains behavior long after the request itself has been handled. A liquidity reservation taken against a nostro account blocks other payments from that account until it is released. An AML hold on a sanctioned-entity match stops a payment from progressing regardless of whether every downstream service is healthy. Once a settlement is finalized, it cannot be rolled back, only compensated. These are not implementation details – they are causal factors in how a failure in one stage cascades to others, and they have no analogue in a generic web service or content-delivery pipeline.

Existing cascading-failure root-cause-analysis (RCA) work treats microservices generically. Soldani et al. [1] explain cascading failures declaratively but reason over service dependencies, not domain state. Recent 2025–2026 systems push RCA accuracy substantially: DynaCausal [2] reaches an average AC@1 of 0.63 by modeling time-varying spatio-temporal dependencies across logs, traces, and metrics; KRCA [3] reaches AC@1 0.88 in production at Kuaishou via a multi-agent causal-graph pipeline. Neither models transactional/domain state as a causal factor – both reason over infrastructure-level signals (latency, error rate, trace topology) that a payment-specific fault can leave completely undisturbed. The closest prior work, ServiceGraph-FM [4], targets payment systems directly and models anomaly propagation as diffusion over the payment-service dependency graph with causal attention over multi-hop paths – but this is still a *structural* model of which services call which; it does not represent liquidity-lock, AML-hold, idempotency, or settlement-finality state as first-class causal evidence. This paper's central claim is that this gap is not cosmetic: Section VII shows state-agnostic methods, including a topology-aware heuristic and a general-purpose LLM given the same telemetry evidence, are actively misled

on exactly the incident class where a louder downstream symptom masks the true upstream root cause – and that payment-state evidence recovers the correct diagnosis where telemetry alone cannot.

Contributions:

- A live, running 8-service ISO 20022 payment pipeline (Section III) instrumented so that liquidity-lock, AML-hold, idempotency, and settlement-finality state is queryable per payment, not just logged as an implementation detail.
- A fault-injection harness that triggers *real* faults against this live system – process crashes, real AML holds, real idempotency collisions – rather than synthesizing labeled ground truth, producing cascades whose propagation, severity, and affected-payment counts are measured post-hoc from real Elasticsearch/Kafka evidence, not asserted in advance (Section V).
- A payment-state-aware RCA method evaluated against a state-agnostic topology/telemetry baseline and a general-purpose LLM baseline given identical evidence, using AC@k with Wilson confidence intervals and a held-out validation split disjoint from the incidents used to develop the method (Section VII) – reporting the result honestly, including a self-audit of methodological weaknesses found and fixed during evaluation (Section IX).

II. Related Work

A. Cascading Failure RCA in Microservices

Soldani et al. [1] explain cascading failures in microservice applications declaratively from a static dependency model – general-purpose, domain-agnostic, and, notably for the fault taxonomy in Section V, not built to distinguish a genuine upstream root cause from a louder downstream symptom under backpressure. RCAEval [5] (735 failure cases, 3 systems, 11 fault types) and its fault-propagation-aware successor [6] (1430 validated failures, 25 fault types, hierarchical/impact-driven ground truth) established AC@k/Avg@k as the standard evaluation protocol for this line of work; Section VII adopts AC@k directly rather than inventing a new metric, while departing from both benchmarks' methodology in one respect worth stating plainly: neither evaluates against a real, running system triggering real faults – both, like the vast majority of RCA benchmarks including this paper's own earlier synthetic pass, generate labeled ground truth rather than measuring it from a live incident. Fang et al. [6] report a finding this paper's own evaluation independently reproduces in a different form: on 4 widely-used public RCA benchmarks, simple rule-based methods match or beat sophisticated SOTA models, because benchmark and evaluation design can make sophistication look like an advantage when it is not actually being tested by the benchmark. Section VII's LLM baseline shows the same pattern – a general-purpose language model given identical evidence to a formulaic heuristic scores *worse* than the heuristic, not better. DynaCausal [2] and KRCA [3] represent the current state of the art on generic microservice telemetry (average AC@1 0.63 and 0.88 respectively, the latter in production), and both attribute their gains to modeling *temporal* and *structural* dynamics more faithfully – richer signal within the same evidence class this paper's `loudest_metric_baseline` and `graph_topology_baseline` occupy (Section VI), not a different evidence class. None of this line of work represents transactional/domain state as a causal factor.

Closest prior art. ServiceGraph-FM [4] targets payment-system RCA directly: a pretrained graph model combining masked graph autoencoding, temporal relational diffusion over the service-dependency graph, and causal attention over multi-hop paths to separate likely causes from correlated downstream effects. This is the paper closest in domain and in intent to distinguish root cause from symptom under propagation – but it is still a model of *which services call which and how anomaly propagates structurally*, learned from graph topology and telemetry. It does not represent liquidity-lock reservation state, AML-hold state, idempotency-key collision state, or settlement-finality state as inputs at all. Section VII's central empirical result is that this specific gap is exploitable: on incidents deliberately constructed so a downstream service's telemetry anomaly is louder than the true upstream root's own signal, a topology/diffusion-style structural model has exactly the failure mode ServiceGraph-FM's own causal-attention mechanism is designed to resist – and payment-state evidence, which no structural model in this line of work consumes, is what actually resolves it.

B. Synthetic Payment and AML Transaction Simulation

PaySim [7] derives payment amount and timing distributions from real aggregated mobile-money statistics and injects labeled fraud without using real entity identities. IBM AMLworld [8] is agent-based and typology-driven (fan-in/fan-out, structuring, layering) with an explicit anonymization and ethical-use stance. The generator in Section IV is grounded methodologically in this line of work and, notably, was not originally built that way: an earlier internal version used real corporate names and real sanctioned-individual names drawn from public sanctions lists for realism. Identifying and correcting this during development, rather than after, is stated here directly as a design correction rather than omitted – it strengthens the methodology section, not weakens it, and is the same anonymization discipline AMLworld's ethics stance establishes as the norm for this kind of dataset.

C. Payment System Operational Resilience

The Bank for International Settlements' Core Principles for Systemically Important Payment Systems [9] and the IMF's more recent survey of operational resilience in digital payments [10] establish that a disruption at a single participant or processing stage can have consequences well beyond that stage – the policy-level statement of the same structural fact this paper's fault taxonomy exercises algorithmically. This body of work is positioned deliberately as policy and continuity-level, not competing with the algorithmic RCA literature surveyed above: it motivates *why* diagnosing payment-pipeline failures

quickly matters at a systemic level, while this paper addresses *how* to diagnose them correctly once a failure has occurred.

Note on a retracted citation: an earlier literature pass surfaced a candidate paper ("Han, latency-aware payment RCA via heterogeneous graph attention," *Computer Science Bulletin*) that was carried as tentative prior art. A direct web search (2026-08-27) for this title, venue, and author combination returns no matching paper, indexed venue, or author/institution trail – it does not check out and is not cited. Its role as "closest prior art" is filled instead by ServiceGraph-FM [4] above, which is independently verified (resolves via DOI on MDPI).

III. System Architecture

ClearFlow is a five-service event-driven payment pipeline built on Spring Boot 3.3.2 / Java 21, coordinated via Apache Kafka: `gateway` → `validation-enrichment` → `aml-compliance` → `routing-execution` → `settlement`, with `fraud-scoring` and `audit` as parallel, non-blocking consumers of the initiation event. Payments are submitted as ISO 20022 pacs.008-shaped requests (`instructionId`, `endToEndId`, `uetr`) and processed through 18 Kafka topics with per-stage idempotency (Redis-backed) and an outbox relay at the gateway.

`fraud-scoring` is a real LightGBM classifier (11 engineered features: amount, temporal, country/currency risk, and real-time transaction-velocity signals) served over HTTP with a circuit-breaker fallback to a heuristic scorer, not a synthetic stub – but it is explicitly not a research contribution of this paper. It exists only to produce realistic fraud-adjacent payment state (`fraudScore`, `riskBand`) as a precondition for fault scenarios, keeping this paper's claim scoped to payment-state-aware RCA rather than fraud detection itself. For completeness: the deployed model is trained on a public mobile-money fraud dataset (PaySim) with genuine, extreme class imbalance (186 fraud examples in 400,000 training rows) and reaches AUC 0.62 on a held-out test set – modest but honest for this rarity. A retrain attempt on a richer dataset (more positive examples, real transaction-velocity signal) reached AUC 0.86 by that metric alone, but was found, before deployment, to produce a badly miscalibrated, near-bimodal score distribution on realistic inputs (roughly half of simulated ordinary payments scored above 0.99), and was rolled back rather than shipped – reported here as a concrete illustration that a single ranking metric does not certify a model fit for a live decisioning threshold.

Three transactional-state mechanisms are central to this paper's RCA contribution:

- **Liquidity reservation** – `routing-execution` takes a pessimistic lock (`SELECT FOR UPDATE`) against a nostro account balance for the duration of routing and settlement.
- **AML holds** – `aml-compliance` screens against a sanctions/PEP list and can block a payment from progressing, independent of downstream service health.
- **Idempotency and settlement finality** – retries are deduplicated by idempotency key; once `SETTLEMENT_COMPLETE` is emitted, the transaction is final and cannot be rolled back, only compensated.

[TODO: Add architecture diagram (Fig. 1): service graph + Kafka topics + state-holding points. Verified live end-to-end 2026-08-17: single payment traced gateway→validated→AML-cleared→SEPA_INSTANT routed→settled, sub-second, correlation-ID traceable across all 5 services – use this trace as the walkthrough example in the figure caption.]

IV. Synthetic Payment and Fault Data Generation

A synthetic corpus generator produces the fast-iteration dataset used alongside the live evaluation of Section VII. An earlier version of this generator used real corporate and sanctioned-individual names for realism; this was identified as a liability during development and replaced entirely with fully synthetic entity identifiers, following the anonymization precedent set by IBM AMLworld [8] – no real company, individual, or sanctioned-entity name appears anywhere in the generator or its output.

Scale. 11,000 persistent debtor/creditor account profiles (country, home currency, risk tier, PEP flag, behavioral baseline) support 50,000 generated payments and a derived causal event timeline of approximately 438,000 state-transition events (8.76 events/payment on average). Rail selection is conditioned on currency (SEPA rails carry only EUR, FEDWIRE/FEDACH/CHIPS only USD, etc.) and each rail enforces a real amount ceiling at selection time, following PaySim-style [7] log-normal amount distributions with type-specific medians. AML scenarios use AMLworld-style typology labels (structuring, layering, integration); an approximately 1% embargo-corridor injection rate exercises the same compliance logic the live system's AML-hold gate implements (Section III), and idempotency-collision rows are generated by cloning an existing payment's debtor/amount/creditor fields and recomputing the same SHA-256 idempotency key the live gateway derives (Section III), so the synthetic dataset's idempotency semantics match the live system's exactly rather than approximating them.

Incident corpus. On top of the payment corpus, controlled incidents are injected across the same four fault families used in the live evaluation (infra, payment-domain, cross-domain, confounded), balanced across fault types rather than left to occur at whatever rate they would naturally, with hierarchical ground truth (root service, component, causal event, propagation path/depth, severity) and temporal-difficulty labels. Confounded incidents are constructed so a downstream symptom service's metric deviation is quantitatively larger than the true root's own – verified directly against the generated data, not merely asserted by design, for every confounded incident in the corpus. Ground-truth incident metadata and the incident-to-payment linkage used for scoring are written to files kept separate from the evidence surface any RCA method reads, so a method cannot trivially read its own answer key; a dedicated

graph-construction check verifies programmatically that no evidence tier a method is restricted to ever exposes a ground-truth node or edge, rather than relying on the separation being followed by convention.

V. Fault Injection and RCA Ground Truth

An earlier version of this evaluation used a synthetic corpus generator producing labeled ground truth by construction (payment/account distributions grounded in PaySim [7] and AMLworld [8], fault injection with hand-set propagation paths and severities). That corpus remains useful for fast iteration and is reported for comparison in Section VII, but its ground truth is generated, not measured – an evaluation on it alone cannot distinguish "the method works" from "the method recovers exactly the assumptions the generator encoded." This section describes the live fault-injection harness built to close that gap: it triggers *real* faults against the running system of Section III and derives ground truth from what actually happens, not from a specification.

A. Fault taxonomy and live-triggerability

Ten fault types across the four families standard to this evaluation protocol (infra, payment-domain, cross-domain, confounded) are triggered through two real mechanisms:

- **Process crash** (infra, cross-domain, confounded families): an admin endpoint kills and cold-restarts the target service by port (`lsof|kill -9`, relaunch the jar), producing a genuine outage under concurrent live traffic. Confounded incidents pair a crash on the true root service with sustained traffic through a service positioned to show backpressure as a louder, correlated symptom – e.g. a `validation-enrichment` crash upstream of `gateway`, which absorbs the resulting queueing/retry pressure and can show a larger raw telemetry deviation than the service that actually failed.
- **Payment-domain triggers**: an AML hold is triggered by submitting a payment against a real sanctions-list entity (confirmed via the live compliance-review endpoint, not assumed); an idempotency collision is triggered by resubmitting an accepted payment's exact payload within the gateway's deduplication window (confirmed via the real HTTP 409 response).

Two fault types specified in an earlier design pass were found, empirically, not to be live-triggerable through the running system as built, and are excluded from the live corpus rather than simulated: a settlement-finality violation is intercepted by an idempotency guard and a database existence check before the finality logic it is meant to test ever executes, and a liquidity-lock-stuck fault requires a hand-constructed Kafka-level message for which no REST path currently exists. Reporting this honestly is itself part of the contribution: a fault taxonomy validated only on paper, without checking whether each fault type is actually reachable through the system under test, risks silently testing the generator's assumptions rather than the system's real behavior.

B. What ground truth is known vs. measured

Because the harness chooses which fault to trigger and when, `fault_type`, `root_service`, and `injection_time` are known with certainty – this is the one property live triggering shares with a synthetic generator. Everything else that would be asserted by a generator is instead measured post-hoc from real evidence: `n_affected_payments` is counted from real payment submissions inside the incident window; `propagation_depth` is measured as the number of services (beyond the root) whose real error-rate z-score exceeds a 2σ deviation from their own pre-incident baseline during the window; `severity` is derived from the observed affected-payment count, not chosen in advance. This measurement is itself imperfect and its limitations are stated directly in Section IX rather than left implicit.

A methodological hazard specific to running many fault injections against one live system in sequence is worth naming explicitly, because it produced a real bug during this work: the pre-incident baseline window used to compute z-scores must not overlap another incident's own effects, or every "quiet baseline" is contaminated by nearby injected faults. Incidents are therefore spaced with an enforced cooldown exceeding the baseline lookback window, and this spacing is logged and checked, not assumed.

VI. RCA Method

Three methods are compared, each explicitly restricted to a named evidence tier so that an accuracy difference can be attributed to what a method is allowed to see, not to one method simply having more information. None of the three ever reads ground-truth incident labels or the incident-to-payment linkage held out for scoring.

A. Baseline: telemetry and topology

`loudest_metric_baseline` ranks services purely by real-time error-rate anomaly: for each service, a z-score of its error rate during the incident window against its own pre-incident baseline (evidence tier G2 – telemetry only, no topology, no payment state). This is the "find the biggest spike" baseline every RCA benchmark in Section II implicitly competes against, and the one a confounded incident is specifically constructed to defeat.

`graph_topology_baseline` (tier G0–G2) adds pipeline topology as a tie-breaker: when two services' z-scores fall within a bounded margin of each other, the more upstream service in the fixed pipeline order is preferred. Topology is deliberately restricted to breaking close calls, not overriding a clear telemetry signal – an earlier, more aggressive version that let topology override z-score rank unconditionally was found to score *worse* than telemetry alone on confounded incidents (a hard upstream-preference rule is wrong whenever backpressure makes a downstream symptom louder, which is exactly what a confounded incident constructs), and is documented here as a negative result rather than discarded silently.

B. Payment-state-aware RCA

`payment_aware_rca` (tier G0–G3) adds the transactional-state mechanisms of Section III as evidence: for payments active during the incident window, the fraction showing an AML hold, a stuck liquidity reservation, a detected idempotency collision, a failed settlement, or (a generalization applying to every crash-based fault, since a process crash touches no payment's own state fields) having stalled before reaching a given pipeline stage's completion event entirely. A domain-state fraction elevated above a fixed threshold is treated as *decisive* – ranked ahead of telemetry – rather than as an additive bonus on top of the z-score. This distinction matters mechanically: a fixed additive term cannot reliably outrank a z-score gap that scales with incident severity, which is precisely what a confounded incident's louder downstream symptom does at high severity; only a decisive override is structurally capable of correcting it. With no elevated payment-state signal, the method falls back to `graph_topology_baseline`'s ranking unchanged.

Two implementation-level findings, surfaced by tracing specific misclassified incidents against real (not synthetic) evidence rather than by tuning against an aggregate accuracy number, are reported here because they generalize beyond this specific system: first, a state-flag defined by a single evidence snapshot (payment reserved and not yet settled) is not specific evidence of a stuck fault when a system has genuine settlement latency – it is also the normal appearance of any payment that simply has not finished yet, and must be gated by how long the payment has actually been observed in that state, not by its state at one instant. Second, this generalizes directly to the "never reached a stage at all" case: a crash-based fault produces no state anomaly whatsoever in the schema this paper's system tracks, and the only signal available is whether a payment's expected completion event for a given stage occurred within a reasonable multiple of that stage's normal latency. Both findings were verified not to change the earlier, generator-based evaluation's results at all (Section VII), which is itself evidence they correct a real gap between synthetic and live evidence rather than fitting noise in the live sample they were derived from.

C. LLM baseline

A fourth method, evaluated at the same G0–G2 evidence tier as `graph_topology_baseline`, tests whether a general-purpose language model reasoning over the same topology and telemetry text outperforms the formulaic heuristic. The model is given the pipeline order, each service's z-score, and an explicit description of the backpressure/loud-symptom failure mode confounded incidents are constructed to test, then asked to rank all five services. This is a controlled comparison of reasoning strategy given identical evidence, not yet a claim about whether an LLM given richer (G0–G3) evidence would do better – that comparison is addressed directly by the next two methods, which isolate the evidence axis and the reasoning axis from each other rather than changing both at once.

D. Static LLM with G0–G3 evidence

A fifth method holds reasoning mode fixed (the identical static, one-shot prompt structure as the G0–G2 baseline) and changes only the evidence tier: the prompt additionally includes the same six payment-domain state fractions Section VI's formula computes (AML hold, liquidity dwell, idempotency, settlement failure, validation retry/stall), presented as text, with no tool access. Compared directly against the G0–G2 LLM baseline, this isolates whether richer evidence alone – independent of how it is used – improves LLM-based diagnosis.

E. Agentic RCA

A sixth method addresses the reasoning axis directly, with real tool access rather than a richer static prompt: given the same G0–G2 topology/telemetry context as the LLM baseline, the model is also given two tools backed by the live MCP gateway's real per-payment endpoints – retrieve a payment's full stage-by-stage timeline, or its AML screening detail – for any of the payments active in the incident window, and may call them iteratively (up to 4 rounds) before answering. This is a materially different agent design than a static prompt: the model decides for itself which of the window's payments are worth inspecting and what to look at, rather than being handed a precomputed summary. MCP's aggregate endpoints (systemic health, active alerts) are deliberately excluded as tools: they aggregate over a window counted back from wall-clock query time, not from the incident's own historical timestamp, and would mix in unrelated later activity for incidents queried hours after they occurred – only the per-payment endpoints are historically exact regardless of when the agent runs.

VII. Evaluation

A. Protocol

AC@1/AC@3 are reported per the RCAEval convention [5], with one deliberate departure justified in Section VI: AC@5 is omitted, since this system has exactly 5 candidate services and AC@5 is therefore 1.0 for every method by construction, not an evaluation result. Every AC@k number is reported with a Wilson 95% confidence interval, and family-level and depth-level strata are reported with their n directly alongside the mean rather than as a bare fraction. Methods are compared via exact McNemar paired significance testing on per-incident AC@1 hit/miss; a comparison is reported only when at least 10 discordant pairs exist, and is explicitly marked as insufficient otherwise rather than implying a precision the sample does not support.

The headline result uses a genuine held-out split: the 17 live incidents collected while the dwell-time gate and stage-stall generalization (Section VI) were being developed and diagnosed against specific misclassified cases form a frozen *dev set*; only the 25 incidents collected *after* both fixes were finalized in code are scored for the comparison below. Two reference-floor methods are scored alongside the four real methods: `random_baseline` (uniform random ranking, AC@1 expectation 0.2) and `majority_baseline` (always

guess the most common root service, fit only on the disjoint dev set).

TABLE I
ROOT CAUSE IDENTIFICATION ACCURACY, HELD-OUT LIVE INCIDENTS ($n = 25$, DEV-SET-DERIVED, NEVER USED TO DEVELOP THE METHOD)

Method	infra (n=12)	pay.-dom. (n=6)	cross-dom. (n=4)	**AC@1 (n=25)**
random_baseline	0.17	0.17	0.25	0.16
majority_baseline	0.25	0.50	0.50	0.32
llm_rca_baseline (G0–G2, static)	0.25	0.50	0.25	0.28
llm_rca_g3_baseline (G0–G3, static)	0.25	0.67	0.00	0.28
loudest_metric (G2)	0.58	0.33	0.50	0.44
graph_topology (G0–G2)	0.58	0.50	0.50	0.48
agentic (G0–G3, LLM+MCP tools)	0.58–0.67	0.50	0.50	0.44–0.52
payment-aware (G0–G3, ours)	**0.67**	**0.83**	0.50	**0.60**

Confounded ($n = 3$) is omitted from the table above and discussed separately: every method, including `random_baseline`, scores 0.0 on the held-out confounded incidents – reported in full in Section VII-C.

B. Held-out result

`payment_aware_rca` reaches overall AC@1 0.60 (95% CI 0.41–0.77) on the held-out set, ahead of `graph_topology_baseline`'s 0.48 (CI 0.30–0.67) and `loudest_metric_baseline`'s 0.44 (CI 0.27–0.63). McNemar paired testing against `random_baseline` reaches significance ($p = 0.0034$, 13 discordant pairs) – the only comparison in this evaluation with enough discordant pairs to clear the 10-pair reporting threshold set in Section VII's protocol. The comparisons against `loudest_metric_baseline` (6 discordant pairs), `graph_topology_baseline` (5), and `llm_rca_baseline` (8) all fall below that threshold; the point estimates favor `payment_aware_rca` in every case, but no significance claim is made for any of them. `payment_domain` is the strongest and most consistent result (0.83 vs. 0.50 and 0.33) – the family the payment-state signals were specifically built to disambiguate, and the family where the observed effect size is least likely to be an artifact of the small held-out sample.

The LLM baseline result from an earlier (dev-set) pass replicates on genuinely fresh data: 0.28 overall, below *both* non-LLM baselines given identical G0–G2 evidence. This is the one dev-set finding that held up unchanged on held-out incidents, and is accordingly the result this paper trusts most among the four methods' comparisons.

Evidence vs. reasoning: a clean, direct answer. `llm_rca_g3_baseline` – the identical static prompting strategy as the G0–G2 baseline, given the same G0–G3 payment-state fractions the formula uses – scores *identically* overall: 0.28 vs. 0.28. Handing a static LLM the exact same richer evidence a formula uses to reach 0.60 produces *zero net change*. This is not uniformly null underneath, which is itself informative: by family, the added evidence helps `payment_domain` (0.50 → 0.667, exactly the family those fractions are literally about) and actively hurts `cross_domain` (0.25 → 0.00, where the added dense numeric text appears to distract more than inform), and these effects cancel to an identical pooled result at $n = 25$ – reporting the family breakdown rather than only the pooled number is what surfaces this instead of averaging it away. Tool access, by contrast, improves broadly and specifically on `infra` (0.25 → 0.58–0.67): given the same information but the reasoning mode to actively decide what to inspect rather than passively receive it, the agent recovers most of the gap to the formulaic method. Together these results answer the question directly: `payment_aware_rca`'s advantage over the LLM-based methods is not explained by access to richer evidence – the static LLM had identical evidence and did not improve – it is explained by the formula's decisive, structured use of that evidence (Section VII-D), which neither static prompting nor currently-scoped agentic tool-use fully replicates.

Tool access closes most of the gap to the formulaic method. Given the same G0–G2 context as the static LLM baseline plus real tool access to the live system, the agentic method reaches 0.44–0.52 overall AC@1 (two repeated runs on the identical held-out set differ by 0.08 despite temperature 0 – unlike every formulaic method, which is bit-for-bit reproducible on the same input, the agentic method's real-time tool-call round-trips introduce genuine run-to-run variance, itself worth stating rather than averaging away). That is more than double the static LLM baseline's 0.28 and matches or exceeds both non-payment-state baselines, purely from letting the model decide what real evidence to look at rather than reasoning over a fixed telemetry summary – while still trailing `payment_aware_rca`'s 0.60. On `infra` specifically the agent ties the payment-aware method exactly (0.67 vs. 0.67).

Dollar-exposure weighting surfaces a real gap, confounded by fault duration. Weighting each held-out incident's AC@1 hit/miss by the real dollar value of payments active in its window (extracted from live Elasticsearch logs, illustrative FX to USD) drops `payment_aware_rca`'s exposure-weighted AC@1 to 0.33 – well below its unweighted 0.60, and the largest such gap of any method tested. Investigated before reporting it as "fails on large transactions": sorting held-out incidents by exposure shows a clean bimodal split ($900K–$5.5M for every crash-mechanism incident vs. $4K–$47K for every payment-domain-trigger incident), which traces directly to injection duration (20–30s crash faults accumulate far more concurrent payments than the 5s payment-domain triggers) rather than to transaction size specifically – and correlates with `confounded`/`cross_domain` already being the hardest families independent of dollar value. The honest reading: there is a real accuracy gap by fault family that dollar-weighting surfaces starkly, but confirming it is specifically about transaction size (not duration) needs exposure normalized per second of window rather than summed over it – not built here, and stated as an open question rather than an unqualified finding either way.

C. The confounded result does not replicate

An earlier analysis pass on the dev set found `payment_aware_rca` recovering strongly on confounded incidents (0.667 AC@1, beating both baselines) after the stage-stall generalization was added. On the 3 confounded incidents in the held-out set, every method – including `random_baseline` – scores 0.0. This is reported exactly as observed, not smoothed over: it is precisely the failure mode a held-out split exists to catch, and it means the confounded-family improvement observed during development does not demonstrably generalize. Whether this reflects a real, family-specific limitation of the current signal set or is simply too small an n (3 incidents) to conclude anything is left genuinely undetermined rather than resolved either way – a larger confounded-specific live sample is the direct next step this result implies.

D. Ablation: which mechanism actually drives the result

`payment_aware_rca` combines two mechanisms (Section VI): 6 payment-domain-specific state fractions (AML hold, liquidity dwell, idempotency, settlement failure, validation retry/stall) and a generalized 5-stage completion-stall signal added specifically to recover accuracy on crash-based faults. Three ablations isolate each mechanism's actual contribution on the same 25 held-out incidents, rather than reporting the combined method as a black box:

TABLE II
ABLATION, HELD-OUT AC@1 ($n = 25$)

Variant	AC@1
fracs-only (stall signal disabled)	0.60
stall-only (all 6 fracs disabled)	0.40
no dwell-time gate (pre-fix behavior)	0.56
full method	**0.60**

The result is more specific, and more useful, than "the method works": **fracs-only exactly matches the full method (0.60 = 0.60)** – the generalized stall signal, despite producing the dev-set's headline confounded recovery (Section VII-C), contributes nothing measurable beyond the payment-domain fractions on genuinely unseen incidents. **stall-only alone underperforms even `graph_topology_baseline`** (0.40 vs. 0.48), meaning the generalization is not simply weaker than the fracs – on its own it is actively worse than not using payment-state evidence at all. This directly explains, rather than merely correlates with, why confounded did not replicate: the specific mechanism responsible for that dev-set result is the one ablation shows does not generalize. The dwell-time gate ablation confirms that fix is real but more modest on this held-out set than its dev-set diagnosis suggested (0.56 vs. 0.60) – expected, since the incident it was traced from was itself in the dev set, not held out.

The paper's validated, mechanism-isolated claim is therefore narrower and more defensible than the headline number alone suggests: payment-domain transactional state (Section III's liquidity/AML/idempotency mechanisms), not process-crash-derived stage-completion evidence, is what this evaluation actually supports as a causal RCA signal.

VIII. Experiment 2: State-Conditioned Failure Propagation

Experiment 1 (Sections VII) asks a diagnostic question: given a cascade, which service caused it. This section asks a different, interventional question that only a live system with real controllable faults can pose at all: does the *same* infrastructure fault produce a *different* cascade depending on what real, persistent payment-domain state already exists in the system when it hits. Every prior-art system in Section II, ServiceGraph-FM included, reasons over passive, already-collected telemetry – none can run this comparison, because none control the intervention.

A. Design

The same fault (`DB_TIMEOUT` on `settlement`) is triggered under two conditions. **Condition A (clean background)** reuses the 5 `DB_TIMEOUT` incidents already collected for Experiment 1 as-is – not re-triggered, since Experiment 1's own data already is this fault type's clean-background condition. **Condition B (AML-held background)**: 2 real AML holds are triggered first and deliberately left unresolved – genuinely persistent live state (a held payment stays `HOLD` until the compliance resolve endpoint is explicitly called, which this experiment never does), not a synthetic label – then the same `DB_TIMEOUT` fault is triggered while those holds remain active, repeated for 4 incidents. Both conditions are measured on the identical pipeline used throughout Experiment 1 (real error-rate z-scores, `measure_propagation_depth`, real per-payment amount/currency for dollar exposure) so no separate, ad-hoc metric definitions are introduced for this comparison.

B. Result: an honest null finding

TABLE III
EXPERIMENT 2: CONDITION A VS. B, RAW VALUES (N TOO SMALL FOR SIGNIFICANCE TESTING)

Metric	Condition A ($n = 5$)	Condition B ($n = 4$)
n_affected_payments (mean)	15.0	14.5
propagation_depth (mean)	1.00	1.00
exposure_usd (mean)	$3.75M	$4.32M

At this sample size, a persistent AML-hold background does not detectably change how a settlement database failure cascades. `propagation_depth` is identically 1 across every single incident in both conditions. `n_affected_payments` ranges overlap almost completely (11–19 vs. 13–16). `exposure_usd` ranges overlap heavily despite Condition B's mean running somewhat higher; at $n = 4$–5 this is not distinguishable from noise, and no significance test is attempted – fabricating one this sample cannot support would undercut the methodological discipline the rest of this paper is built on.

Two readings are stated, and neither is asserted as the answer. This could be a real, operationally reassuring finding: the system's fault isolation held, and an unrelated compliance hold elsewhere did not amplify an unrelated infrastructure failure's blast radius. Or $n = 4$–5 per condition may simply be too small to detect a real effect, compounded by `propagation_depth`'s own measurement ceiling – this live system's crash-recovery is fast enough that depth rarely exceeds 1 for any infra fault under either condition (Section VII), limiting how much room this specific metric has to show a state-conditioning effect even if one exists. A larger Condition-B sample and a fault type whose depth already varies more under clean conditions are the concrete next steps this null result implies. Reporting a carefully-checked null result from an experiment no passive-dataset methodology in this literature could even run is more useful to the field than a convenient positive result would have been.

IX. Discussion

A. Threats to validity

Sample size. The live evaluation in Section VII runs at a small number of incidents per fault family. Every result is reported with a Wilson 95% confidence interval specifically so this is not obscured by a bare mean; a family-level accuracy difference of one or two incidents is not distinguishable from chance at this n, and is reported as such rather than as a finding. A larger live corpus, not a synthetic one, is the direct next step this limitation implies, since the entire motivation for building the live harness (Section V) was that synthetic ground truth cannot answer whether a method's synthetic-benchmark result transfers.

Tautological ground truth for crash-based faults. For every infra/cross-domain/confounded incident, `root_service` is, by construction, the process the harness chose to kill. This is real infrastructure and real telemetry noise, but it is *not* genuine diagnostic ambiguity of the kind a real operator faces when the root cause is actually unknown – the harness always knows the answer it is testing against. This is the same limitation present in essentially all fault- injection-based RCA benchmarks, including the ones surveyed in Section II, and is stated plainly here rather than allowed to be implied by the word "real."

Threshold selection. The payment-state elevation threshold and the dwell-time gates in Section VI are domain-motivated constants justified by measured normal-operation latencies (e.g. observed real settlement completion in milliseconds), not values fit or cross-validated against held-out data. A genuine train/validation split for these specific constants, distinct from the incident-level dev/held-out split already used for the headline comparison, is future work once the live corpus is large enough to support it without starving the held-out test set.

Single-organization topology. The pipeline models one organization's internal payment processing, not a multi-participant/ consortium settlement network where root cause can originate outside the observed system entirely. The payment-state mechanisms evaluated here (liquidity locks, AML holds, idempotency, finality) are internal to a single processor's pipeline; whether they generalize to a network-level topology is untested.

Measured, not asserted, propagation depth. `propagation_depth` is itself a z-score-threshold heuristic computed from the same telemetry the RCA methods consume, not an independently established ground truth. Depth-stratified results in Section VII should be read as descriptive of what the same measurement pipeline reports, not as evidence of a causally verified depth effect.

B. Why report this at all

Every limitation above was found by directly auditing this paper's own evaluation methodology after an initial round of results looked cleaner than the sample size could support – an earlier draft of Section VII's live result reported a family-level accuracy comparison derived from incidents whose baseline windows were, it turned out, contaminated by adjacent fault injections, and a fix validated only against the same incidents used to diagnose it. Both were caught and corrected before this version, not smoothed over, and the corrected evaluation pipeline (Wilson CIs, a genuine held-out split, reference floor baselines, paired significance testing) is the pipeline Section VII actually reports. This is stated explicitly because a systems paper whose own author found and disclosed these gaps is a stronger, not weaker, paper than one where a reviewer finds them first.

X. Conclusion

Payment-specific transactional state – liquidity locks, AML holds, and idempotency collisions – is a causal factor in how failures cascade through a payment pipeline, and existing microservice RCA methods, general or LLM-based, do not model it. On real incidents triggered against a live, running system and scored against a held-out set disjoint from the incidents used to develop the method, a payment-state-aware RCA method reaches AC@1 0.60, ahead of both a topology-aware and a telemetry-only baseline, and significantly ahead of random guessing. An ablation, not just the headline number, is what makes this claim defensible: the payment-domain state fractions alone reproduce the full method's result exactly, while a generalized process-crash completion-stall signal contributes nothing beyond them and underperforms alone – the paper's validated contribution is specifically payment-domain transactional state, not a general theory of crash-fault evidence, and the confounded family's dev-set improvement not replicating on held-out data is the direct, explained consequence of that same finding rather than an unrelated gap. One implementation lesson generalizes beyond this system regardless of that scope: a single-snapshot state flag is not specific evidence of a stuck fault without a dwell-time gate, verified to matter on live data (0.56 vs. 0.60 without it) while leaving the synthetic benchmark's numbers completely unchanged – evidence it corrects a real synthetic-to-live gap rather than fitting noise in the sample it was derived from. Extending payment-domain-specific state evidence to the fault

families it does not yet cover, rather than generalizing the crash signal further, and extending the LLM baseline to the same G0–G3 evidence tier the payment-aware method uses, are the direct next steps this evaluation's own ablation points to.

A separate, interventional experiment (Section VIII) – possible only because the evaluation system is live and controllable, not a passive dataset – found no detectable difference in cascade propagation between two real, controlled payment-state backgrounds for the fault type tested. That null result is reported with its own limitations stated directly, not smoothed over: a larger sample and a fault type with more room to show a state-conditioning effect are the concrete next steps it implies, and the experiment's own value is that no prior-art system in this literature could have run it at all.

REFERENCES

[1] J. Soldani *et al.*, "What went wrong? explaining cascading failures in microservice-based applications," in *Proceedings of the International Conference on Service-Oriented Computing*. Springer, 2021, verify full citation details (pages, exact author list) before submission.

[2] S. Zhang, A. Fang, Y. Yang, R. Cheng, X. Tang, and P. He, "Dynacausal: Dynamic causality-aware root cause analysis for distributed microservices," *arXiv preprint arXiv:2510.22613*, 2025, average AC@1 0.63 on RCAEval-RE2-OB and a new large-scale benchmark; generic microservice signals (logs/traces/metrics), no domain-state modeling.

[3] J. Jiang, J. Feng, Y. Luo, Q. Zhang, Y. Sun, W. Gu, S. Zhang, T. Cui, Y. Wu, J. Huang, N. Qi, and D. Pei, "Krca: An efficient root cause analysis system in hyper-scale microservice systems via agentic ai," *arXiv preprint arXiv:2607.01788*, 2026, aC@1 0.88 (root-cause service) / 0.79 (failure-type classification); deployed at Kuaishou, 77.3% diagnosis-time reduction. Agentic multi-stage pipeline over generic metrics/causal-graph priors, not payment-domain state.

[4] Z. Zeng and M. Zhou, "Servicegraph-fm: A graph-based model with temporal relational diffusion for root-cause analysis in large-scale payment service systems," *Mathematics*, vol. 14, no. 2, p. 236, 2026, closest verified prior art: payment-system RCA via a pretrained graph model (masked graph autoencoding + temporal relational diffusion + causal attention over multi-hop paths). Distinguish explicitly in Section II: they model payment-SERVICE topology/anomaly propagation structurally, not payment-STATE semantics (AML holds, liquidity locks, idempotency, settlement finality) as causal factors – the gap this paper's contribution fills.

[5] Q. Pham *et al.*, "Rcaeval: A benchmark for root cause analysis of microservice systems," *arXiv preprint arXiv:2412.17015*, 2024, 735 failure cases, 3 systems, 11 fault types; AC@k/Avg@k metrics – adopt this evaluation protocol.

[6] A. Fang, S. Zhang, Y. Yang, H. Wu, J. Xu, X. Wang, R. Wang, M. Wang, Q. Lu, and P. He, "Rethinking the evaluation of microservice rca with a fault propagation-aware benchmark," *arXiv preprint arXiv:2510.04711*, 2025, 1430 validated failures, 25 fault types; hierarchical/impact-driven ground truth. Directly relevant finding: simple rule-based methods match or beat SOTA models on 4 widely-used public RCA benchmarks – cite alongside this paper's own llm_rca_baseline result (Section VII), where a general-purpose LLM given the same evidence as a formulaic heuristic scores WORSE than the heuristic. Same underlying caution: benchmark/evaluation design can make sophistication look like it's winning when it isn't.

[7] E. Lopez-Rojas, A. Elmir, and S. Axelsson, "Paysim: A financial mobile money simulator for fraud detection," *European Modeling and Simulation Symposium (EMSS)*, 2016.

[8] E. Altman *et al.*, "Realistic synthetic financial transactions for anti-money laundering models," *NeurIPS Datasets and Benchmarks Track / arXiv:2306.16424*, 2023, iBM AMLworld; explicit ethical-use/anonymization statement – cite this as the anonymization precedent in Section IV.

[9] Committee on Payment and Settlement Systems (CPSS), Bank for International Settlements, "Core principles for systemically important payment systems," Bank for International Settlements, Tech. Rep. CPMI Papers No. 43, 2001. [Online]. Available: https://www.bis.org/cpmi/publ/d43.pdf

[10] T. Khiaonarong, H. Leinonen, and R. Rizaldy, "Operational resilience in digital payments: Experiences and issues," International Monetary Fund, Tech. Rep. WP/21/288, 2021.